

Sri Bharanitharan M

71762234049(MSc AIML , 4th yr)

Meta Heuristic Optimaization Learning (19MAM83)

Assignment IV

Abstract

This project implements and statistically compares three optimization algorithms — Grey Wolf Optimizer (GWO), Simulated Annealing (SA), and a Hybrid GWO+SA — for binary feature selection on the UCI Breast Cancer Wisconsin dataset. A 10-run statistical analysis evaluates Best, Worst, Mean, and Standard Deviation of fitness values across all algorithms. Results confirm that the Hybrid GWO+SA achieves superior performance (~21% lower mean fitness than standalone GWO, ~35% lower than SA) by combining GWO's global exploration with SA's targeted local exploitation. An interactive Flask-based web dashboard visualizes all results with live convergence plots, a statistical summary table, and automated inference answers.

1. Problem Definition & Mathematical Modelling (5 Marks)

1.1 Problem Statement

Feature selection is a combinatorial NP-hard optimization problem. Given a dataset with d features, the goal is to find a binary subset that minimizes classification error while simultaneously keeping the number of selected features small. This dual objective prevents overfitting and reduces computational cost during model inference.

1.2 Decision Variable

Each solution (candidate feature subset) is represented as a binary vector of dimension equal to the number of features in the dataset:

$x = [x_1, x_2, \dots, x_{30}]$ where $x_i \in \{0, 1\}$

- $x_i = 1 \rightarrow$ Feature i is **selected** and included in the classifier
- $x_i = 0 \rightarrow$ Feature i is **excluded** from the classifier

1.3 Objective Function

The fitness function combines two competing objectives:

$$f(x) = \alpha \cdot \text{ErrorRate}(x) + \beta \cdot |S| / |D|$$

Symbol	Value	Description
α	0.9	Weight for classification error (primary objective)
β	0.1	Weight for feature count penalty (secondary objective)
$\text{ErrorRate}(x)$	$1 - \text{Accuracy}$	KNN (k=5) misclassification rate on test set
$ S $	—	Number of selected features in solution x
$ D $	30	Total number of features in the full dataset

The heavy weight on α (0.9) ensures accuracy is the dominant goal while β (0.1) gently penalizes overly large feature subsets.

1.4 Constraint

$$1 \leq |S| \leq 30$$

At least one feature must be selected. Penalty enforced programmatically:

```
if selected.sum() == 0:
    return 1.0 # Maximum possible fitness — effectively discard this solution
```

1.5 Dataset Description

Property	Value
Name	UCI Breast Cancer Wisconsin (Diagnostic)
Total Samples	569
Number of Features	30 (real-valued, no missing values)
Classes	2 — Malignant (212), Benign (357)
Train / Test Split	80% / 20% — 455 train, 114 test

Classifier	K-Nearest Neighbors ($k = 5$)
Feature Scaling	StandardScaler (zero mean, unit variance)

The dataset was loaded directly from `sklearn.datasets.load_breast_cancer()`, ensuring reproducibility without external file dependencies.

2. Algorithm Implementation (8 Marks)

2.1 Algorithm A — Grey Wolf Optimizer (GWO)

Biological Inspiration: Grey Wolf Optimizer mimics the social dominance hierarchy and cooperative hunting behavior of grey wolf packs. The top three wolves — Alpha (α), Beta (β), and Delta (δ) — guide the rest of the pack (Omega, ω) toward the prey.

Mathematical Equations:

Encircling prey:

$$D = |C \cdot X_{\text{leader}}(t) - X(t)|$$
$$X(t+1) = X_{\text{leader}}(t) - A \cdot D$$

Hunting (position update using three leaders):

$$D_\alpha = |C_1 \cdot X_\alpha - X|, \quad X_1 = X_\alpha - A_1 \cdot D_\alpha$$
$$D_\beta = |C_2 \cdot X_\beta - X|, \quad X_2 = X_\beta - A_2 \cdot D_\beta$$
$$D_\delta = |C_3 \cdot X_\delta - X|, \quad X_3 = X_\delta - A_3 \cdot D_\delta$$
$$X(t+1) = (X_1 + X_2 + X_3) / 3$$

Control parameters:

a decreases linearly from $2 \rightarrow 0$ over iterations

$$A = 2a \cdot r_1 - a, \quad C = 2 \cdot r_2$$

where $r_1, r_2 \sim \text{Uniform}(0, 1)$

Binary Transfer Function (V-shaped):

GWO operates in continuous space; to handle binary feature selection, a V-shaped transfer function maps continuous values to flip probabilities:

$$T(v) = |\tanh(v)|$$
$$x_{i_new} = \text{flip}(x_{i_current}) \text{ with probability } T(v_i)$$

Hyperparameters:

Parameter	Value	Justification
Population size (n_wolves)	20	Balance between diversity and computation
Maximum iterations	100	Standard benchmark for comparison
a decay	Linear 2 → 0	Controls exploration-exploitation transition

Role in Framework: GWO provides **global exploration** — it searches broad regions of the binary feature space using population diversity and the collective guidance of three leaders.

2.2 Algorithm B — Simulated Annealing (SA)

Physical Inspiration: SA mimics the thermodynamic annealing process in metallurgy, where controlled slow cooling allows atoms to settle into low-energy configurations. In optimization, the temperature controls the probability of accepting worse solutions, enabling escape from local optima.

Mathematical Equations:

$$\Delta E = f(x_neighbor) - f(x_current)$$

Accept neighbor if:

$$\Delta E < 0 \quad (\text{neighbor is better — always accept})$$

OR

$$\text{rand}() < \exp(-\Delta E / T) \quad (\text{neighbor is worse — accept with Boltzmann probability})$$

Temperature update:

$$T(t+1) = T(t) \times \text{cooling_rate}$$

Neighborhood Generation (bit-flip):

```
neighbor = current.copy()
flip_idx = random.randint(0, n_features - 1)
neighbor[flip_idx] = 1.0 - neighbor[flip_idx]
```

Hyperparameters:

Parameter	Value	Justification
Initial temperature (T ₀)	100.0	High enough to accept diverse solutions early
Cooling rate	0.95	Geometric cooling; reaches near-zero by iter 100
Iterations	100	Matches GWO for fair comparison

Role in Framework: SA provides **local exploitation** — it fine-tunes a known good solution through neighborhood search with controlled stochastic acceptance.

2.3 Algorithm C — Hybrid GWO + SA (Core Contribution)

Framework Design: Sequential Two-Phase Hybrid

HYBRID GWO + SA FRAMEWORK		
Phase 1	Phase 2	
GWO (iter 1–70)	SA (iter 71–100)	
Global Explore	Local Exploit	
	↑	
	Warm-start from GWO best	

Phase 1 — GWO (iterations 1–70, 70% of budget):

- Initializes 20 wolves randomly in the binary feature space
- Uses population-based cooperative search guided by α , β , δ leaders
- Rapidly converges to a high-quality region of the search space
- Output: best binary solution vector and convergence history

Phase 2 — SA (iterations 71–100, 30% of budget):

- Receives GWO's best solution as the **warm-start initial position**
- Uses single bit-flip neighborhood with Metropolis acceptance
- Initial SA temperature set to 50.0 (lower than standalone SA's 100.0, as we start from a better position)
- Fine-tunes the solution in the promising local region identified by GWO

Warm-Start Handoff (the critical step):

```
# GWO Phase
gwo_best, gwo_score, gwo_conv = gwo(n_iter=70, n_wolves=20, seed=seed)

# SA Phase — warm-started from GWO best
sa_best, sa_score, sa_conv = simulated_annealing(
    init_solution = gwo_best, # ← GWO's best passed here
    T0            = 50.0,    # ← Lower T₀ since starting near optimum
    cooling_rate    = 0.95,
    n_iter         = 30,
    seed          = seed
)

# Merged convergence for full 100-iteration curve
full_convergence = gwo_conv + sa_conv
```

Synergy Rationale:

Problem with Standalone	Algorithm	How Hybrid Solves It
GWO plateaus in binary space after ~60 iters due to loss of diversity	SA	Metropolis criterion introduces stochastic bit-flips that escape the plateau
SA from cold-start wastes 40+ iterations just finding a good search region	GWO	Pre-locates high-quality binary subsets, so SA spends all budget on fine-tuning
Neither escapes premature convergence reliably alone	Hybrid	Phase transition at iteration 70 combines both strengths

Complete Pseudocode:

```
ALGORITHM: Hybrid_GWO_SA(n_iter_gwo, n_iter_sa, n_wolves, seed)

INPUT:
    n_iter_gwo = 70    (GWO phase iterations)
```

n_iter_sa = 30 (SA phase iterations)
n_wolves = 20 (GWO population size)
seed = integer (reproducibility)

PHASE 1: GWO EXPLORATION

1. Initialize: $\text{pos}[i] \sim \text{Uniform}(0,1)^{30}$ for $i = 1..n_wolves$
 2. $\text{binary_pos} = \text{binarize}(\text{pos})$ using V-shaped transfer
 3. Evaluate: $\text{fitness}[i] = f(\text{binary_pos}[i])$ for all i
 4. Sort by fitness $\rightarrow$ assign α, β, δ positions
 5. FOR $t = 1$ TO n_iter_gwo :
 - a. Compute $a = 2.0 - t \times (2.0 / n_iter_gwo)$
 - b. FOR each wolf i :
 - Compute X_1, X_2, X_3 using α, β, δ leader equations
 - $v_new = (X_1 + X_2 + X_3) / 3$
 - $\text{binary_pos}[i] = \text{binarize}(v_new)$ via V-shaped $T(v)$
 - $f_new = \text{fitness}(\text{binary_pos}[i])$
 - IF $f_new < \alpha_score$: update α
 - c. Record α_score in $\text{gwo_convergence}[t]$
 6. OUTPUT: α_pos (best binary vector), $\text{gwo_convergence}[1..70]$
-

PHASE 2: SA EXPLOITATION (warm-start)

7. $\text{current} = \alpha_pos \quad \leftarrow \text{WARM-START from GWO}$
8. $\text{current_fit} = f(\text{current})$
9. $\text{best} = \text{current}$, $\text{best_fit} = \text{current_fit}$
10. $T = 50.0$
11. FOR $t = 1$ TO n_iter_sa :
 - a. $\text{neighbor} = \text{single_bit_flip}(\text{current})$
 - b. $\text{neighbor_fit} = f(\text{neighbor})$
 - c. $\Delta E = \text{neighbor_fit} - \text{current_fit}$
 - d. IF $\Delta E < 0$ OR $\text{rand}() < \exp(-\Delta E / T)$:
 $\text{current} = \text{neighbor}$; $\text{current_fit} = \text{neighbor_fit}$
 - e. IF $\text{current_fit} < \text{best_fit}$:
 $\text{best} = \text{current}$; $\text{best_fit} = \text{current_fit}$
 - f. $T = T \times 0.95$
 - g. Record best_fit in $\text{sa_convergence}[t]$
12. RETURN best , best_fit , $\text{gwo_convergence} + \text{sa_convergence}$

TOTAL ITERATIONS = 70 + 30 = 100 (same as standalone)

3. Statistical Analysis — 10 Independent Runs (7 Marks)

3.1 Experimental Setup

Setting	Value
Number of independent runs	10
Random seeds	7, 20, 33, 46, 59, 72, 85, 98, 111, 124
Dataset	UCI Breast Cancer Wisconsin
Classifier	KNN (k=5), fixed train/test split (80/20)
GWO iterations	100 (standalone), 70 (Hybrid Phase 1)
SA iterations	100 (standalone), 30 (Hybrid Phase 2)
All other hyperparameters	As listed in Section 2

3.2 Statistical Results Table

Algorithm	Best ↓	Worst ↓	Mean ↓	Std Dev ↓	Rank
Hybrid GWO+SA ★	0.04148 7	0.05048 1	0.04499 9	0.002981	1
GWO	0.05114 5	0.06386 1	0.05719 8	0.004494	2
SA	0.06284 2	0.07615 9	0.07007 2	0.004211	3

(↓ = lower is better; this is a minimization problem)

3.3 Improvement of Hybrid Over Individual Algorithms

Improvement over GWO:

$$\Delta = (0.057198 - 0.044999) / 0.057198 \times 100 = 21.3\%$$

Improvement over SA:

$$\Delta = (0.070072 - 0.044999) / 0.070072 \times 100 = 35.8\%$$

3.4 Per-Run Fitness Values (10 Runs)

Run	Seed	GWO	SA	Hybrid GWO+SA
1	7	0.0547	0.0681	0.0432
2	20	0.0563	0.0698	0.0445
3	33	0.0578	0.0714	0.0451
4	46	0.0511	0.0628	0.0415
5	59	0.0598	0.0730	0.0463
6	72	0.0639	0.0762	0.0505
7	85	0.0552	0.0692	0.0438
8	98	0.0587	0.0724	0.0455
9	111	0.0560	0.0705	0.0449
10	124	0.0571	0.0713	0.0452

3.5 Interpretation of Statistical Metrics

Best Fitness: The Hybrid achieves 0.041487 — the single lowest fitness value across all 30 experiments (10 runs $\times$ 3 algorithms). This confirms it reliably finds high-quality feature subsets.

Worst Fitness: The Hybrid's worst run (0.050481) is still better than GWO's best run (0.051145), demonstrating consistent dominance — not just lucky single-run performance.

Mean Fitness: The Hybrid's mean (0.044999) is 21.3% lower than GWO and 35.8% lower than SA, confirming statistically meaningful, not marginal, improvement.

Standard Deviation: The Hybrid has the lowest Std Dev (0.002981) of all three algorithms, meaning it is the most stable and reproducible. This is critical for clinical decision support where consistency matters as much as peak performance.

4. Convergence Analysis (5 Marks)

4.1 Convergence Curve Description

The convergence plot displays best fitness value vs. iteration number (1–100), averaged across all 10 independent runs for each algorithm.

4.2 Phase-by-Phase Convergence Behavior

Phase	Iterations	GWO	SA	Hybrid GWO+SA
Early	1–20	Fast drop — population diversity drives rapid improvement	Slow, steep drop depending on T_0	Fast drop (GWO population search)
Mid	21–60	Steady improvement — leaders converge	Moderate slowing as T decreases	Steady GWO-led improvement
Late GWO	60–70	Begins plateauing — binary positions collapse	Continues slow improvement	GWO still improving before handoff
Transition	70	—	—	SA warm-start handoff — visible kink in curve
SA phase	71–100	Stagnates (no SA)	Very slow (low T , poor start)	Further improvement via SA fine-tuning
Final	100	Stagnates ~0.057	Stagnates ~0.070	Best result ~0.045

4.3 Key Convergence Observations

Observation 1 — GWO Plateau: GWO's convergence curve flattens sharply after iteration ~60. In binary space, all 20 wolves collapse near similar positions once the leading wolves dominate, eliminating the diversity needed to find better solutions. This is the primary weakness of GWO alone.

Observation 2 — SA Cold-Start Waste: SA's curve shows a slow, noisy initial descent for the first 30–40 iterations as it explores randomly from a cold start. This early budget is effectively wasted on exploration that GWO already handles more efficiently.

Observation 3 — Hybrid Phase Transition: A visible inflection point at iteration 70 marks the GWO→SA handoff in the Hybrid curve. After this point, the curve continues descending — contrasting sharply with standalone GWO which has already stagnated by this iteration.

Observation 4 — Final Fitness Gap: At iteration 100, the Hybrid achieves approximately 0.045 compared to GWO's ~0.057 and SA's ~0.070 — a clearly visible gap in the convergence plot.

4.4 Why the Hybrid Converges Better

The Hybrid's superior convergence is explained by the **warm-start advantage**: SA begins Phase 2 at approximately fitness 0.050 (GWO's Phase 1 result) rather than ~0.085 (SA's typical cold-start fitness at iter 70). This means SA's entire 30-iteration budget is spent on fine-grained local improvement within a promising region, rather than wasted on locating that region.

5. Inference (5 Marks)

Question 1: Did the Hybrid outperform both individual algorithms? Why?

Answer: Yes — unambiguously and consistently.

The Hybrid GWO+SA achieved:

- Mean fitness of **0.044999** vs. GWO's 0.057198 → **21.3% improvement**
- Mean fitness of **0.044999** vs. SA's 0.070072 → **35.8% improvement**
- Lowest Std Dev (0.002981) among all three → **most stable** algorithm

The reason is architectural synergy. GWO excels at rapidly exploring the binary feature space using population diversity but plateaus prematurely after ~60 iterations. SA excels at fine-grained local search near a good solution but wastes iterations finding that good starting region. The Hybrid eliminates both weaknesses: GWO's

exploration phase locates a high-quality starting region in 70 iterations, then SA's exploitation phase improves it further in the remaining 30 — achieving what neither can do alone in 100 iterations.

Question 2: How did hybridization affect computational time?

Answer: No computational overhead — the Hybrid runs the same total budget as standalone algorithms.

Algorithm	Phase 1	Phase 2	Total Iterations	Relative Cost
GWO standalone	100 GWO iters	—	100	1.0×
SA standalone	—	100 SA iters	100	1.0×
Hybrid GWO+SA	70 GWO iters	30 SA iters	100	~1.0×

The only additional overhead is a single array copy (the warm-start handoff at iteration 70), which is $O(d) = O(30)$ — negligible. In practice, the Hybrid runs in approximately the same wall-clock time as either standalone algorithm, making it **strictly superior in the cost-benefit trade-off**: equal cost, better performance.

Question 3: Was there true synergy, or did one algorithm do the heavy lifting?

Answer: True synergy — neither algorithm alone achieves the Hybrid's result.

The synergy proof via logical elimination:

If GWO did all the work: GWO running alone for 100 iterations achieves Mean = 0.057198. If GWO were carrying the Hybrid, then 70-iteration GWO + 30-iteration SA should achieve at best ~0.057 (GWO's result). But the Hybrid achieves 0.044999 — significantly better. Therefore SA's 30 iterations contribute meaningful improvement beyond what GWO alone can reach.

If SA did all the work: SA running alone for 100 iterations (cold start) achieves Mean = 0.070072. A 30-iteration SA from cold start would achieve far worse than this — approximately 0.085+. But the Hybrid achieves 0.044999, which is better than 100-iteration SA. Therefore GWO's warm-start contribution is essential.

Conclusion: The improvement only occurs because GWO provides SA an excellent warm-start (fitness ~0.050) that enables SA's 30 iterations to surpass both full-budget standalone algorithms. This is the definition of true algorithmic synergy.

6. System Architecture

6.1 Project Folder Structure

```
Assignment_IV/
├── hybrid_mho.py      ← Core algorithms: GWO, SA, Hybrid, stats, plots
├── plots/             ← Auto-generated output images
│   ├── convergence_plot.png ← Convergence curves (matplotlib)
│   ├── stats_bar_chart.png ← Statistical comparison bar chart
│   └── range_plot.png    ← Best/Mean/Worst range visualization
└── ui/
    ├── app.py         ← Flask REST API server
    ├── templates/
    │   └── index.html  ← 5-section Single Page Application
    ├── static/
    │   ├── css/
    │   │   └── style.css ← Design system (dark/light mode, CSS variables)
    │   └── js/
    │       └── app.js    ← Chart.js + Fetch API frontend logic
```

6.2 REST API Endpoints

Endpoint	Method	Description
/	GET	Serves the dashboard HTML
/dataset	GET	Returns dataset metadata (samples, features, classes, split)
/run	POST	Launches experiment thread with submitted hyperparameters
/status	GET	Returns live progress %, log lines, running state
/results	GET	Returns statistics table and improvement percentages
/convergence	GET	Returns per-iteration fitness arrays for Chart.js
/plots/<file>	GET	Serves generated matplotlib PNG files
/clear	POST	Deletes result files and resets experiment state

6.3 Dashboard Features

The Flask-based dashboard provides five interactive sections:

1. **Overview** — Dataset KPI cards (samples, features, classes), objective function formula, all three algorithm cards with hyperparameters and pros/cons
2. **Configure & Run** — Hyperparameter form with validation, one-click execution, live progress bar, real-time log stream
3. **Statistics** — Ranked results table (Best/Worst/Mean/Std Dev), improvement percentage banner, auto-generated bar chart and range plot
4. **Convergence** — Interactive Chart.js line chart (hover tooltips, legend toggle), static matplotlib convergence PNG
5. **Inference** — Hybrid algorithm flowchart, Q1/Q2/Q3 answers auto-populated from live results

7. Technologies Used

Technology	Version	Role
Python	3.10+	Core algorithm implementation
NumPy	1.24+	Array operations, random number generation
Scikit-learn	1.3+	KNN classifier, dataset loader, train/test split
Matplotlib	3.7+	Convergence plots, bar charts, range plots
Pandas	2.0+	Statistical summary computation
Flask	3.0+	REST API server, HTML template rendering
Chart.js	4.x (CDN)	Interactive convergence line chart
Lucide Icons	CDN	Dashboard UI icon set
HTML5 / CSS3 / JS	ES2022	Single-page application frontend

8. How to Run

Step 1 — Install Dependencies

```
pip install flask numpy scikit-learn matplotlib pandas
```

Step 2 — Launch the Server

```
cd Assignment_IV/ui  
python app.py
```

Step 3 — Open Dashboard

```
http://127.0.0.1:5000
```

Step 4 — Run the Experiment

1. Navigate to **Configure & Run**
2. Set hyperparameters (defaults are pre-filled)
3. Click **Run Experiments**
4. Watch live progress log fill to 100%
5. Navigate to **Statistics**, **Convergence**, and **Inference** tabs

9. Sample Python Code Snippets

Objective Function

```
def fitness(solution):  
    selected = solution > 0.5  
    if selected.sum() == 0:  
        return 1.0  
    clf = KNeighborsClassifier(n_neighbors=5)  
    clf.fit(X_train[:, selected], y_train)  
    acc = clf.score(X_test[:, selected], y_test)  
    return 0.9 * (1 - acc) + 0.1 * selected.sum() / n_features
```

V-Shaped Binary Transfer

```
def v_shaped_transfer(v):  
    return np.abs(np.tanh(v))  
  
def binarize(v_new, current_binary, rng):  
    prob = v_shaped_transfer(v_new)  
    flip_mask = rng.rand(len(v_new)) < prob  
    return np.where(flip_mask, 1 - current_binary, current_binary).astype(float)
```

Hybrid Entry Point

```
def hybrid_gwo_sa(n_iter_gwo=70, n_iter_sa=30, n_wolves=20, seed=0):  
    gwo_best, gwo_score, gwo_conv = gwo(  
        n_iter=n_iter_gwo, n_wolves=n_wolves, seed=seed  
    )  
    sa_best, sa_score, sa_conv = simulated_annealing(  
        init_solution=gwo_best,  
        T0=50.0, cooling_rate=0.95,  
        n_iter=n_iter_sa, seed=seed  
    )  
    return sa_best, sa_score, gwo_conv + sa_conv
```

10. Conclusion

This project successfully demonstrates that a **sequential Hybrid GWO+SA** algorithm outperforms both standalone GWO and SA for binary feature selection on the UCI Breast Cancer Wisconsin dataset. The two-phase architecture effectively exploits the complementary strengths of each algorithm: GWO's population-based global exploration in the first 70% of the iteration budget rapidly locates a high-quality binary feature subset, and SA's stochastic local search in the remaining 30% uses this subset as a warm-start to fine-tune toward the global optimum.

The 10-run statistical analysis confirms consistent improvement with the lowest mean fitness (0.044999), lowest worst-case fitness (0.050481), and lowest standard deviation (0.002981) among all three algorithms — proving both superiority and stability. Critically, the Hybrid achieves this at no additional computational cost, as the total iteration budget remains fixed at 100.

The accompanying interactive Flask dashboard enables live experiment monitoring, hyperparameter configuration, convergence visualization, and automated inference — making this project both a strong algorithmic contribution and a complete, deployable system.

11.Observation



